Advanced C Course

Code Quality applied to C - Session 5

Pascal Rothnemer

Spring 2026, EPITA

3 March 2026

Session 5 – Content

1)MCQ test

2)Hands-on on projects

3)Course

- Multi-tasking, Parallel Computing, HPC
- Code Refactoring (if time allowed)

MCQ #1 – What is the output

```c
#include <stdio.h>
int main()
{
    int i = 5;
    int k = i % 2;
    printf("%d\n", k);
    return 0;
}
```

A) Compile time error

B) 1

C) 0

D) 2

MCQ #2 – What is the output

```
#include <stdio.h>
int main()
{
    float x = 37.5;
    int k = x;
    printf("%d\n", k);
}
```

A) Compile time error

B) 37

C) 0

D) 38

MCQ #3 – What is the output

```
#include <stdio.h>
void main()
{
    int x = 0;
    int* ptr = &x;
    printf("%p \n", ptr);
    x++;
    printf("%p \n", ptr);
}
```

A) Depends on the compiler

B) Two different addresses

C) Compile time error

D) two identical addresses

MCQ #4

What does the **const** keyword do?

A) Allocates dynamic memory

B) Makes a variable modifiable

C) Defines a function

D) Prevents variable modification

MCQ #5 – What is the output

```
void foo(int* p) {
    *p = 20;
}
int main()
{
    int i = 10;
    foo(&i);
    printf("%d\n", i);
    return 0;
}
```

A) Compile time error

B) 10 20

C) 20

D) 10

MCQ #6 – What is the output

```
#include <stdio.h>

int main()

{

    int a = 20;

    int b = ++a;

    printf("Value of b is %d", b);

}
```

A) Value of b is 22

B) Value of b is 20

C) Value of b is 21

D) Undefined behaviour

MCQ #7 – What is the output

```
#include <stdio.h>

int main()

{

    int a = 3;

    int b = a++;

    printf("Value of b is %d", --b);

}
```

A) Value of b is 3

B) Value of b is 4

C) Value of b is 2

D) Undefined behaviour

MCQ #8 – What is the output

```
#include <stdio.h>
int main()
{
    int i = 0;
    int x = i++, y = ++i;
    printf("%d % d\n", x, y);
    return 0;
}
```

A) 0, 1

B) 0, 2

C) 1, 2

D) Undefined

MCQ #9

What are the different ways to initialize an array with all elements as zero?

A) int array[5] = {};

B) int array[5] = {0};

C) int array[5] = {0, 0, 0, 0, 0};

D) All of the mentioned

MCQ #10

Which of the following header declares mathematical functions and macros?

A) math.h

B) assert.h

C) stdmat. h

D) stdio. h

MCQ #11

What is a function in C++?

A) A reusable block of code

B) A data type

C) A variable

D) A loop

MCQ #12 – What is the output

```
#include <stdio.h>
void main()
{
    float x = 0.1;
    if (x == 0.1)
        printf("equal");
    else
        printf("not equal");
}
```

A) equal

B) not equal

C) Run time error

D) Compile time error

MCQ #13 – What is the output

```c
#include <stdio.h>
int main()
{
    int i = 5;
    i = i / 3;
    printf("%d\n", i);
    return 0;
}
```

A) Implementation defined

B) 1

C) 3

D) Compile time error

MCQ #14 – What is the output

```c
#include <stdio.h>
void main()
{
    int x = 5;
    if (x < 1)
        printf("hello");
    if (x == 5)
        printf("hi");
    else
        printf("no");
}
```

A) hi

B) hello

C) no

D) error

MCQ #15

How do you pass an array to a function in C++?

A) By copying

B) By value

C) By pointer

D) By name

MCQ #16 – What is the output

```
#include <iostream>
using namespace std;
int main() {
    int arr[3] = { 1, 2 };
    for (int i = 0; i < 3; i++)
        cout << arr[i] << " ";
    return 0;
}
```

A) 1 2 3

B) 1 2 1

C) 1 2 0

D) Compilation error

MCQ #17

Which of the following are true about multidimensional arrays in C++?

A) They are arrays of arrays

B) The syntax to declare a 2D array is type arrayName[rows][columns];

C) They can have two or more dimensions

D) All of the above

MCQ #18

Which of the following statements about arrays is incorrect?

A) Arrays are always passed by value to functions

B) The name of the array is a pointer to its first element

C) Arrays can be initialized at the time of their declaration

D) None of the above is correct

MCQ #19 – What is the output

```
#include <iostream>
using namespace std;
int main() {
    int arr[5] = { 1, 2, 3, 4, 5 };
    int* p = arr;
    cout << *(p + 2);
    return 0;
}
```

A)1
B)3
C)4
D)2

MCQ #20

Which access specifier allows members to be accessed only within a C++ class?

A) public

B) protected

C) private

D) none of them

MCQ #21

What is the main benefit of encapsulation in C++?

A) Data security and abstraction

B) Faster execution

C) Better UI design

D) Simplified syntax

MCQ #22

What does **OOP** stand for?

A) Organized Object Programming

B) Object-Oriented Programming

C) Object-Oriented Procedure

D) None of the above

MCQ #23

Which keyword is used to define a class in C++?

A)object

B)struct

C)class

D) None of the above

MCQ #24

What is an object in C++?

A) A type of function

B) A variable

C) An instance of a class

D) None of the above

MCQ #25

Which feature of OOP allows a class to acquire properties of another class?

A) Encapsulation

B) Polymorphism

C) Abstraction

D) Inheritance

CORRECTION

MCQ #1 – What is the output

```
#include <stdio.h>
int main()
{
    int i = 5;
    int k = i % 2;
    printf("%d\n", k);
    return 0;
}
```

A) Compile time error

B) 1

C) 0

D) 2

MCQ #2 – What is the output

```
#include <stdio.h>
int main()
{
    float x = 37.5;
    int k = x;
    printf("%d\n", k);
}
```

A) Compile time error

B) 37

C) 0

D) 38

MCQ #3 – What is the output

```
#include <stdio.h>
void main()
{
    int x = 0;
    int* ptr = &x;
    printf("%p \n", ptr);
    x++;
    printf("%p \n", ptr);
}
```

A) Depends on the compiler

B) Two different addresses

C) Compile time error

D) **two identical addresses**

MCQ #4

What does the **const** keyword do?

A) Allocates dynamic memory

B) Makes a variable modifiable

C) Defines a function

D) Prevents variable modification

MCQ #5 – What is the output

```
void foo(int* p) {
    *p = 20;
}
int main()
{
    int i = 10;
    foo(&i);
    printf("%d\n", i);
    return 0;
}
```

A) Compile time error

B) 10 20

C) 20

D) 10

MCQ #6 – What is the output

```
#include <stdio.h>
int main()
{
    int a = 20;
    int b = ++a;
    printf("Value of b is %d", b);
}
```

A) Value of b is 22

B) Value of b is 20

C) Value of b is 21

D) Undefined behaviour

MCQ #7 – What is the output

```
#include <stdio.h>

int main()

{

    int a = 3;

    int b = a++;

    printf("Value of b is %d", --b);

}
```

A) Value of b is 3

B) Value of b is 4

C) Value of b is 2

D) Undefined behaviour

MCQ #8 – What is the output

```
#include <stdio.h>
int main()
{
    int i = 0;
    int x = i++, y = ++i;
    printf("%d % d\n", x, y);
    return 0;
}
```

A) 0, 1

B) 0, 2

C) 1, 2

D) Undefined

MCQ #9

What are the different ways to initialize an array with all elements as zero?

A) int array[5] = {};

B) int array[5] = {0};

C) int array[5] = {0, 0, 0, 0, 0};

D) All of the mentioned

MCQ #10

Which of the following header declares mathematical functions and macros?

A) math.h

B) assert.h

C) stdmat. h

D) stdio. h

MCQ #11

What is a function in C++?

A) A reusable block of code

B) A data type

C) A variable

D) A loop

MCQ #12 – What is the output

```
#include <stdio.h>
void main()
{
    float x = 0.1;
    if (x == 0.1)
        printf("equal");
    else
        printf("not equal");
}
```

A) equal

B) not equal

C) Run time error

D) Compile time error

MCQ #13 – What is the output

```c
#include <stdio.h>
int main()
{
    int i = 5;
    i = i / 3;
    printf("%d\n", i);
    return 0;
}
```

A) Implementation defined

B) 1

C) 3

D) Compile time error

MCQ #14 – What is the output

```
#include <stdio.h>
void main()
{
    int x = 5;
    if (x < 1)
        printf("hello");
    if (x == 5)
        printf("hi");
    else
        printf("no");
}
```

A) hi

B) hello

C) no

D) error

MCQ #15

How do you pass an array to a function in C++?

A) By copying

B) By value

C) By pointer

D) By name

MCQ #16 – What is the output

```
#include <iostream>
using namespace std;
int main() {
    int arr[3] = { 1, 2 };
    for (int i = 0; i < 3; i++)
        cout << arr[i] << " ";
    return 0;
}
```

A) 1 2 3

B) 1 2 1

C) 1 2 0

D) Compilation error

MCQ #17

Which of the following are true about multidimensional arrays in C++?

A) They are arrays of arrays

B) The syntax to declare a 2D array is type arrayName[rows][columns];

C) They can have two or more dimensions

D) All of the above

MCQ #18

Which of the following statements about arrays is incorrect?

A) **Arrays are always passed by value to functions**

B) The name of the array is a pointer to its first element

C) Arrays can be initialized at the time of their declaration

D) None of the above is correct

MCQ #19 – What is the output

```
#include <iostream>
using namespace std;
int main() {
    int arr[5] = { 1, 2, 3, 4, 5 };
    int* p = arr;
    cout << *(p + 2);
    return 0;
}
```

A)1
B)3
C)4
D)2

MCQ #20

Which access specifier allows members to be accessed only within a C++ class?

A) public

B) protected

C) private

D) none of them

MCQ #21

What is the main benefit of encapsulation in C++?

A) Data security and abstraction

B) Faster execution

C) Better UI design

D) Simplified syntax

MCQ #22

What does **OOP** stand for?

A) Organized Object Programming

B) Object-Oriented Programming

C) Object-Oriented Procedure

D) None of the above

MCQ #23

Which keyword is used to define a class in C++?

A)object

B)struct

C)class

D) None of the above

MCQ #24

What is an object in C++?

A) A type of function

B) A variable

C) An instance of a class

D) None of the above

MCQ #25

Which feature of OOP allows a class to acquire properties of another class?

A) Encapsulation

B) Polymorphism

C) Abstraction

D) Inheritance

ANSWERS

1B
2B
3D
4D
5C
6C
7C
8B
9D
10A
11A
12B
13B
14A
15C
16C
17D
18A
19B
20C
21A
22B
23C
24C
25D

Projects Schedule

1) Today – Session : Hands-on with help on projects
2) March 10th – Session : Hands-on with help on projects
3) Project sent to me by March 16th
4) March 17th : projects presentation / evaluation
5) Results within 1 week

Projects Deliverables

1) Source Code, with supporting readme.txt or equivalent, that Compiles & Runs
   - Comes with needed environment (files, libraries...) if any
2) Documentation – Presentation, discussing :
   - Project scope – reasons for choice
   - Description of development case – difficulties, choices, management of unexpected...
   - Description of features (added if applicable)

Evaluation Criteria

based on a combination of criteria including :

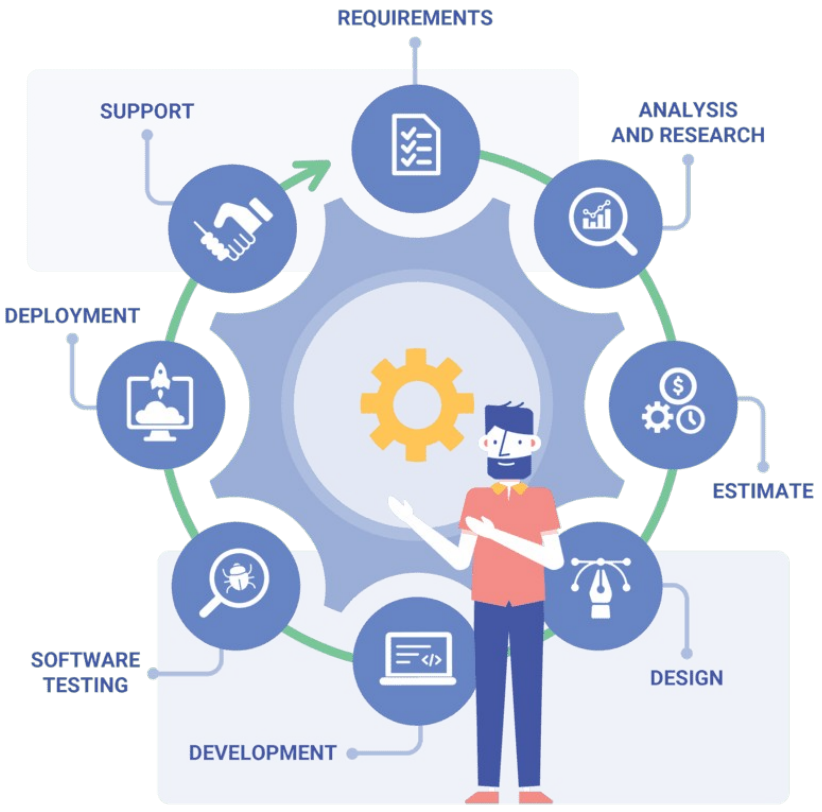
- Attendance & MCQ test
- Features added or created
- presentation & design (including answers to questions)
- code quality (readability, structuration... i.e. **internal** quality)
- code complexity and difficulty

Quote of the day

« Software and cathedrals are much the same : first we build them, then we pray. »

Developing a Software Product

- What is a project ?
- When does a project start ?
- When does a project end ?
- What is a software product ?
- Do we need to manage software development ?

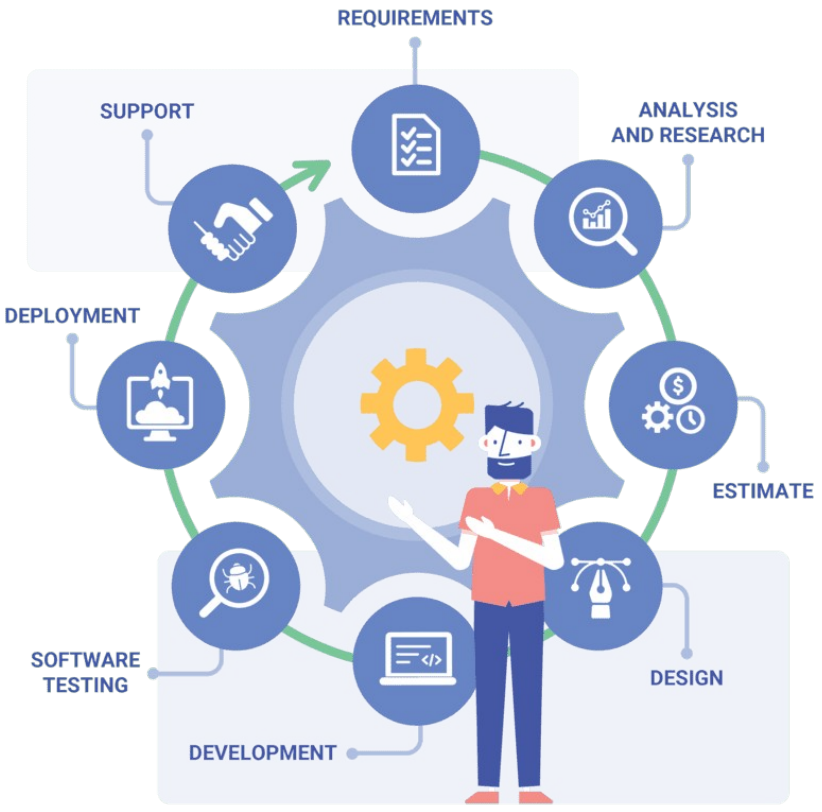


Project vs Product

- **Project** : set of timely defined and humanly assigned activitities needed to provide a « Product »
- **Product** : An application, program, library, system, set of executables, performing tasks as defined by requirements

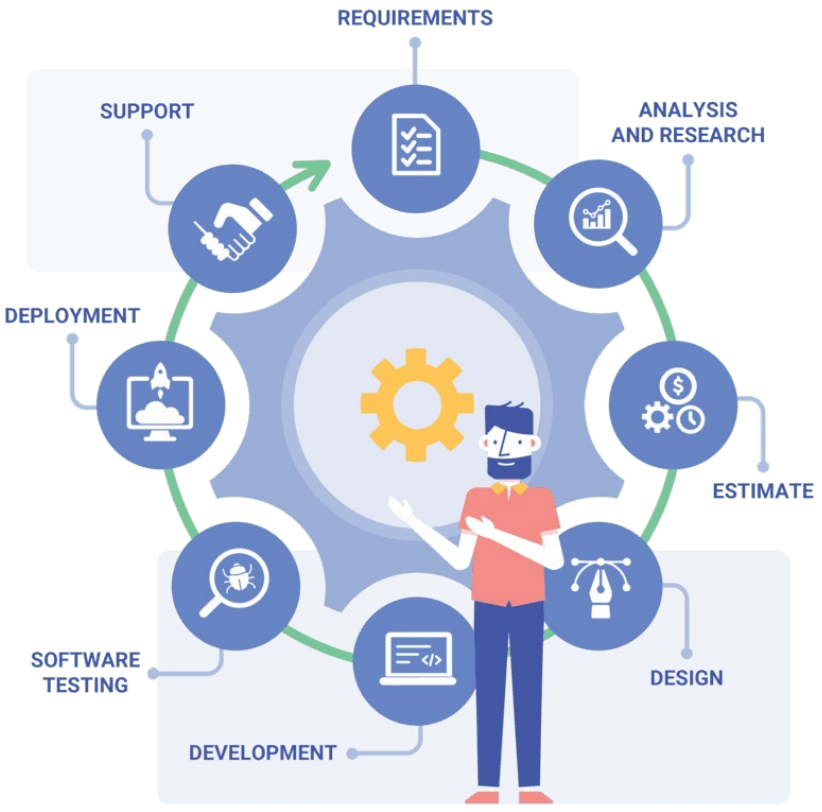
Developing a Software Product

- **Software Product**
  - A software piece developed for the benefit of users to satisfy a need in the market.
- **Project management**
  - Doing things, in a (more or less) organized manner, needed for project execution ; if nothing specific is done, it just means that things will be done in a more chaotic manner



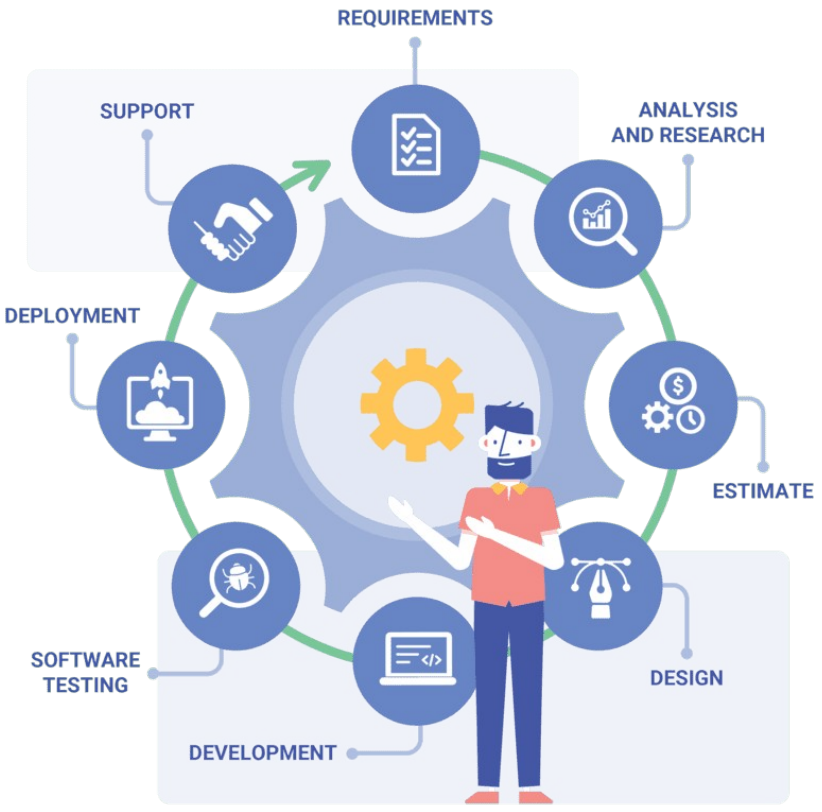
Developing a Software Product

- **Software project**
  - designing and implementing a software solution to satisfy a user's requirement (**order**)



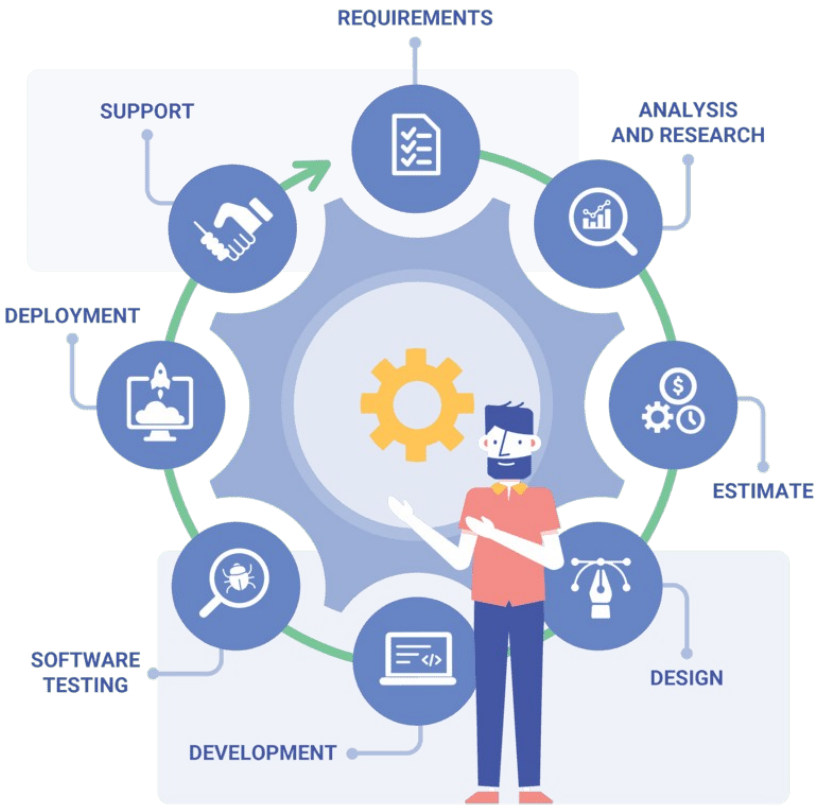
Developing a Software Product

- Project starts when...



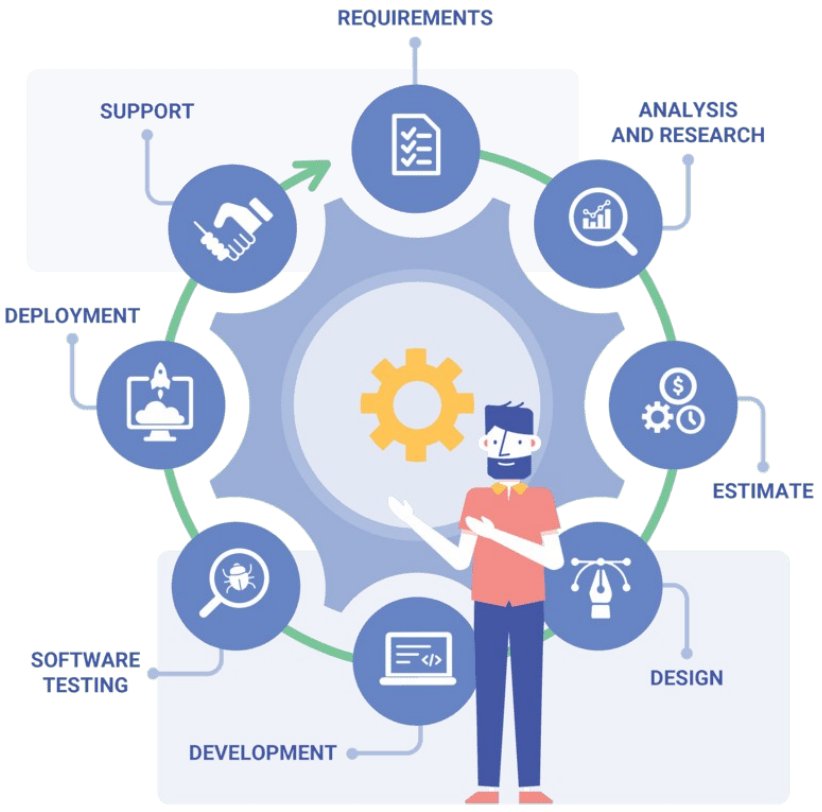
Developing a Software Product

- Project starts when...
  - it has been officially decided, « launched » (and **funded**)



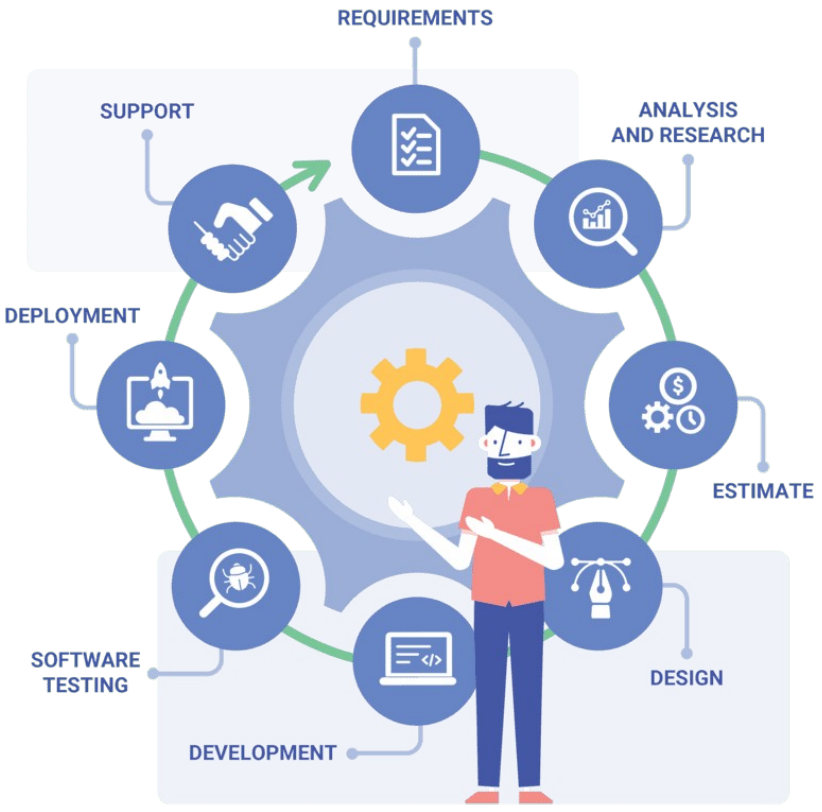
Developing a Software Product

- Project starts when...
  - it has been officially decided, « launched » (and **funded**)
- Project ends when...



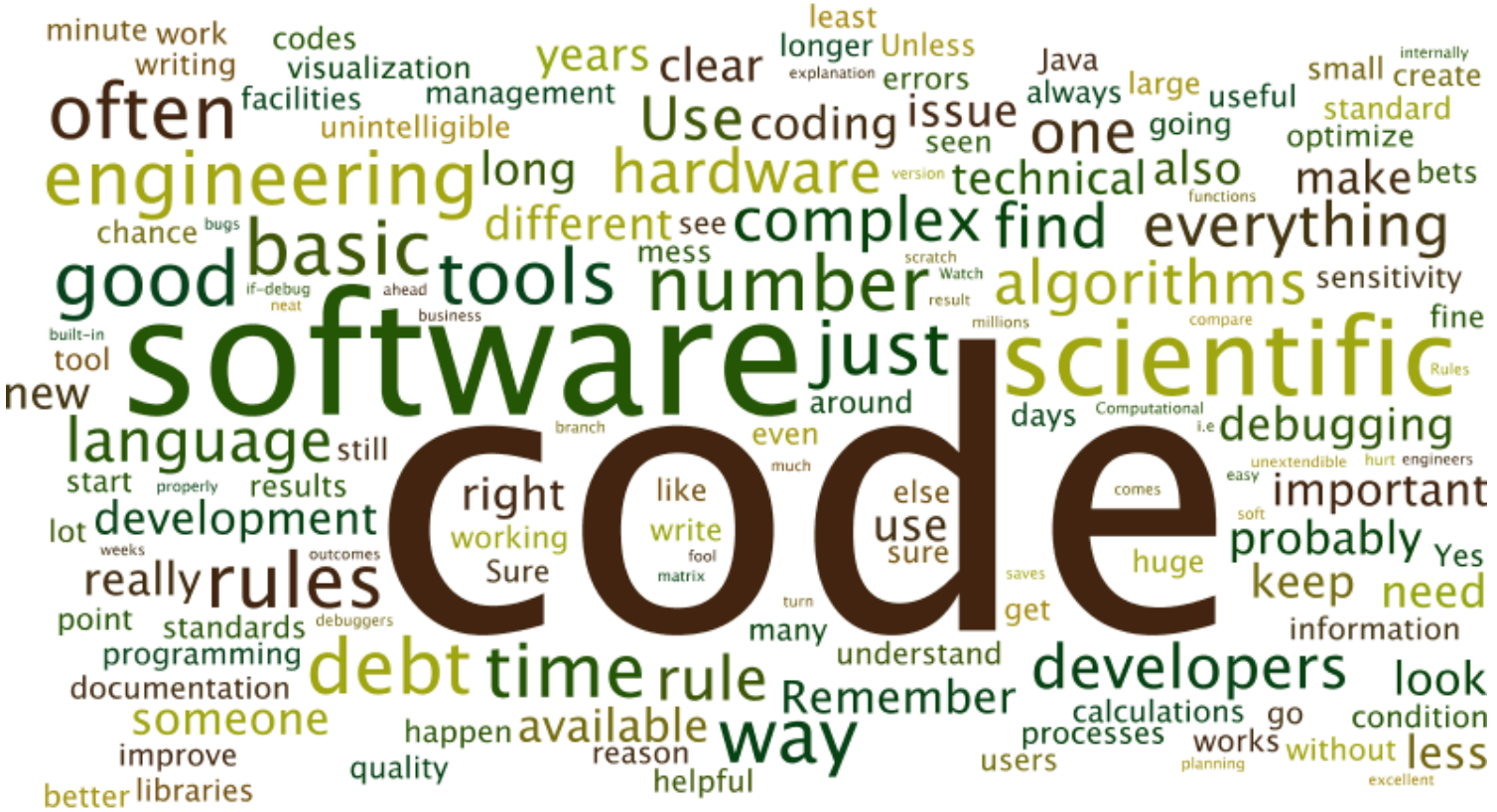
Developing a Software Product

- Project starts when...
  - it has been officially decided, « launched » (and **funded**)
- Project ends when...
  - Goal is achieved
  - Product commercialized
  - Product abandoned
  - ...



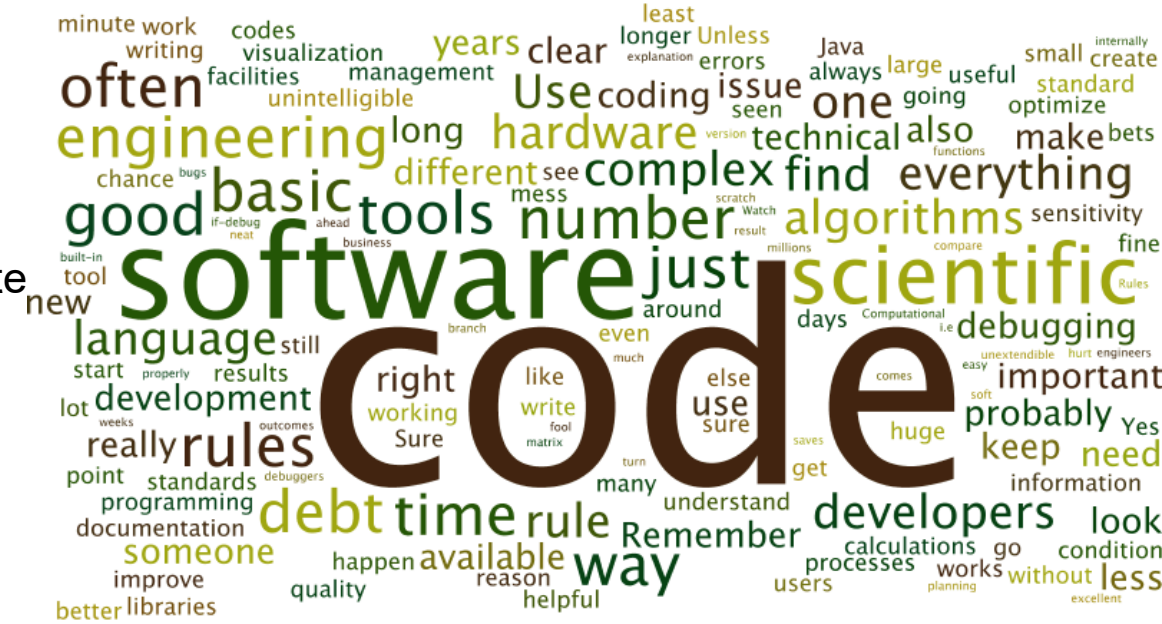
Developing a Software Product

Key words


minute work
codes
writing
visualization
years
clear
least
longer Unless
explanation
errors
Java
small
create
internally
often
facilities
management
Use
coding
issue
always
large
useful
standard
unintelligible
seen
one
going
optimize
engineering
long
hardware
version
technical
also
make
bets
functions
chance
bugs
different
see
complex
find
everything
basic
mess
good
tools
number
scratch
Watch
algorithms
sensitivity
if-debug
ahead
neat
result
business
fine
built-in
software
just
millions
compare
scientific
tool
new
Rules
around
days
Computational
i.e
debugging
language
still
branch
even
much
unextendible
hurt
engineers
easy
start
properly
results
right
like
else
comes
important
lot
development
working
write
use
soft
probably
Yes
weeks
outcomes
Sure
fool
sure
huge
really
rules
matrix
saves
keep
need
point
standards
debuggers
turn
get
many
information
programming
debt
time
rule
understand
developers
look
documentation
Remember
someone
calculations
go
condition
happen
available
processes
works
without
less
improve
way
reason
users
planning
quality
helpful
better
libraries
excellent

Key words - activities

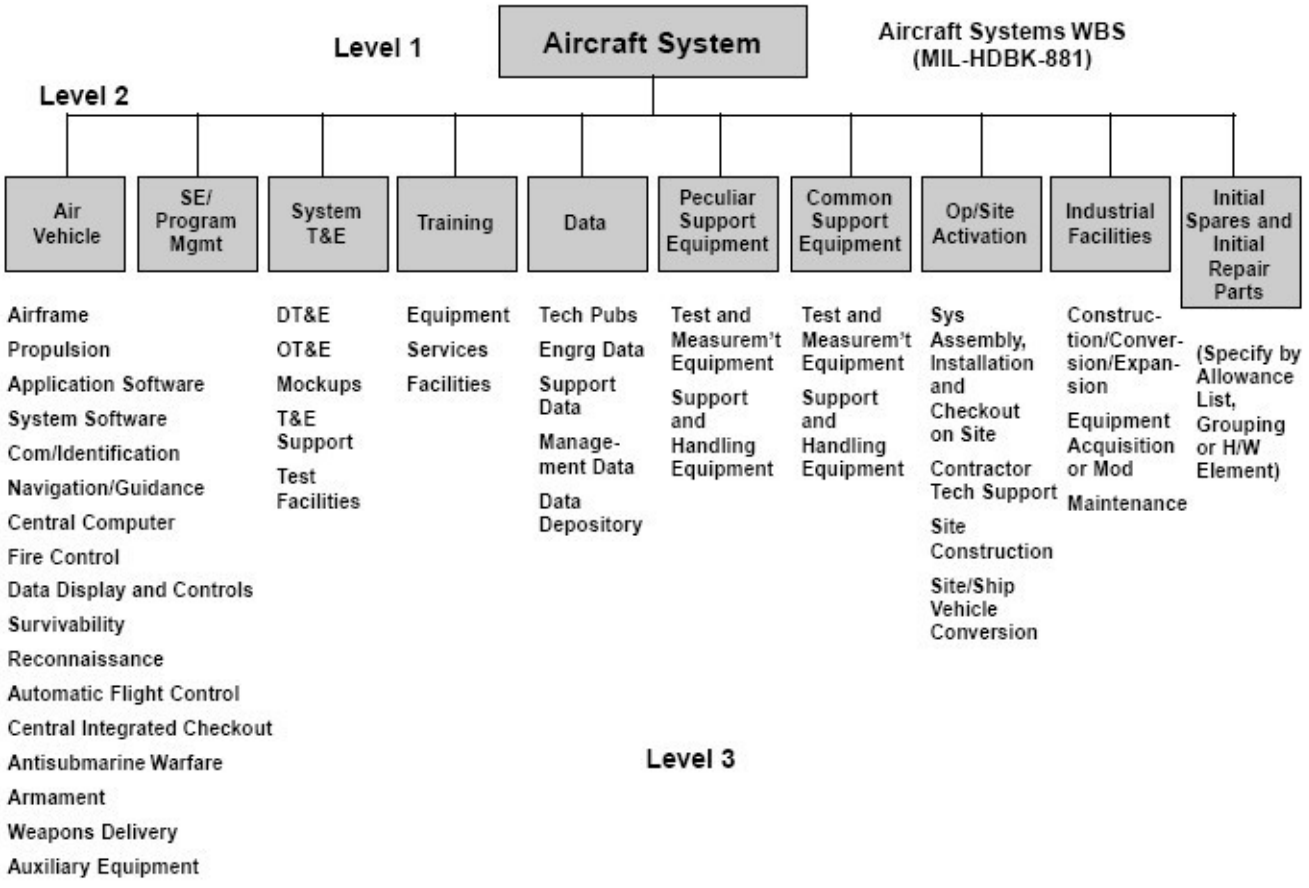
- Gathering Requirements
- Writing Specifications
- Select a Process
- Defining Architecture
- Perform Analysis
- Designing
- **Coding**
- Setting Milestones
- Estimate Cost
- Control Quality
- Defining tasks
- Planning
- Staffing
- Estimate Time
- Estimate Effort
- Calculate Metrics
- Set Cmz Target date
- Testing (Beta)
- Release
- Maintain
- Commercialize
- Declare Obsolescence



Tasks – Tasking – TBS or WBS

- *Task : an activity that needs to be accomplished within a defined period of time or by a deadline to work towards work-related goals.*
- *« Task Breakdown Structure (TBS) or Work Breakdown Structure (WBS) : a key project management element that organizes the team's work into manageable sections»*
- *At WBS stage tasks are listed but not ordered in time nor dependency*

WBS - Example


Level 1
Aircraft System
Aircraft Systems WBS
(MIL-HDBK-881)
Level 2
Air Vehicle
SE/ Program Mgmt
System T&E
Training
Data
Peculiar Support Equipment
Common Support Equipment
Op/Site Activation
Industrial Facilities
Initial Spares and Initial Repair Parts
Airframe
Propulsion
Application Software
System Software
Com/Identification
Navigation/Guidance
Central Computer
Fire Control
Data Display and Controls
Survivability
Reconnaissance
Automatic Flight Control
Central Integrated Checkout
Antisubmarine Warfare
Armament
Weapons Delivery
Auxiliary Equipment
DT&E
OT&E
Mockups
T&E Support
Test Facilities
Equipment
Services
Facilities
Tech Pubs
Engrg Data
Support Data
Manage-ment Data
Data Depository
Test and Measurem't Equipment
Support and Handling Equipment
Test and Measurem't Equipment
Support and Handling Equipment
Sys Assembly, Installation and Checkout on Site
Contractor Tech Support
Site Construction
Site/Ship Vehicle Conversion
Construc-tion/Conver-sion/Expan-sion
Equipment Acquisition or Mod
Maintenance
(Specify by Allowance List, Grouping or H/W Element)
Level 3

Activities invovled and needed

- **Requirements Gathering & Analysis**: Understand problem
- **Writing Specifications** : Define what product will do what
- **Architecture :** Define overall structure of the product
- **Design** : Define modules, sub-modules, Interfaces…
- **Coding** : Start to code, do mockups, prototypes...
- **Integration & Testing** : Validate product components, system
- **Deploy :** Install or provide installers to selected target users
- **Maintainance** : accompagn product during its lifetime

Activities invovled and needed

- Managing the team
- Keeping project on track, manage planning
- Purchasing material, licences...
- Hiring contractors…
- Selecting external components
- Saying Yes or No to proposals (internal, external)

Stakeholders, Actors, Roles

- project's stakeholder : a person involved in the project (internal or external)
- Role : defines specific tasks and responsibilities to be played at a given time by a person to make the project happen and success

- Actor : A person that plays a particular rôle
- A person can perform one or several roles

Some Roles

- Product manager
- Product owner
- Engineering manager
- Software architect
- Software developers
- UX/UI designers
- QA engineer
- Business analyst
- Scrum master
- Testers
- Team lead or Technical lead

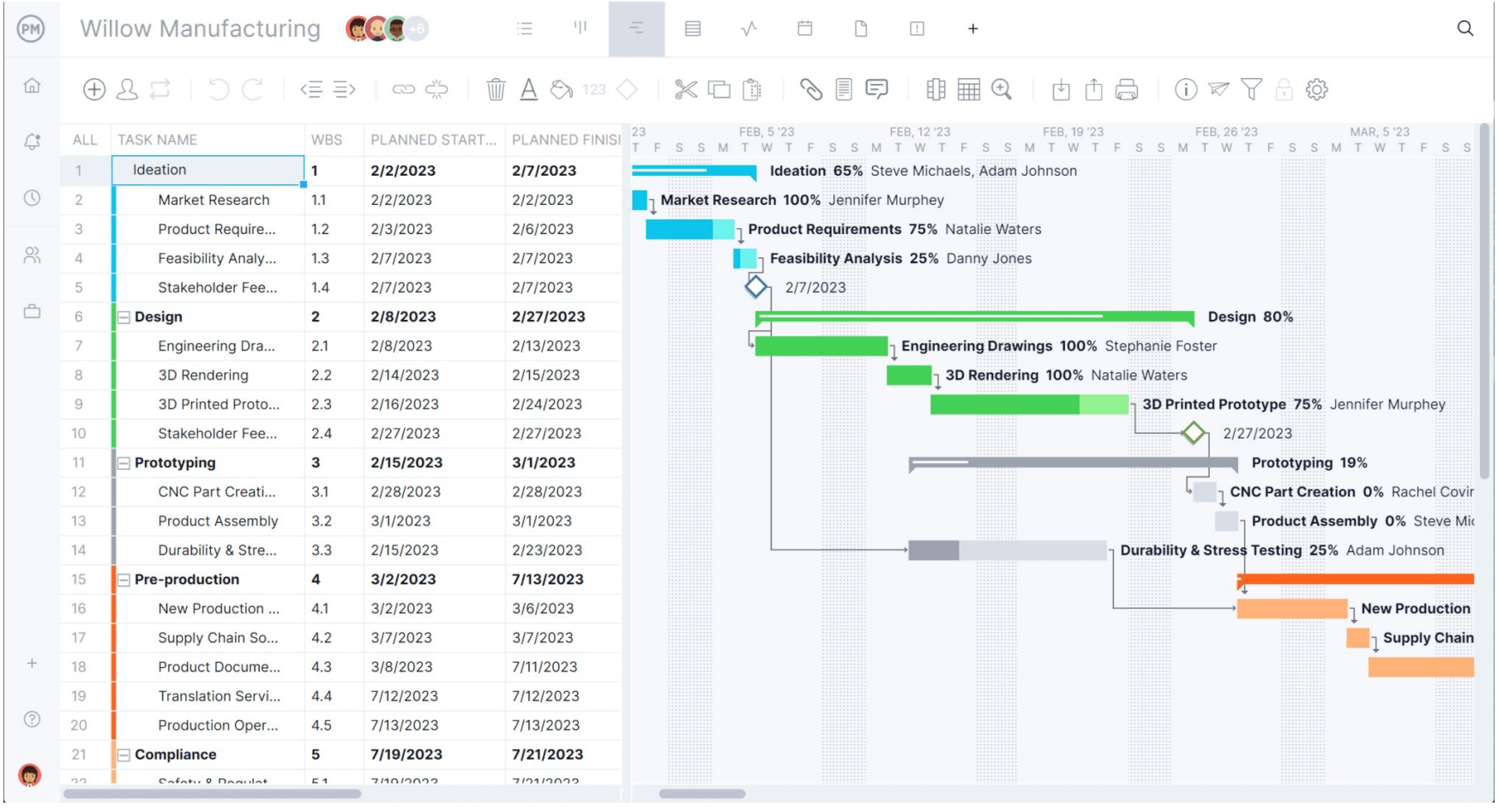
Deliverables & Milestones

- **Deliverable** : a piece of work that is delivered :
  - Documentation, Internal document
  - Prototype
  - Software installer
- **Milestone** : A key date marking a specific development stage
  - Project launched
  - Specifications Document approved
  - Installer of a prototype ready for test...

Project Planning

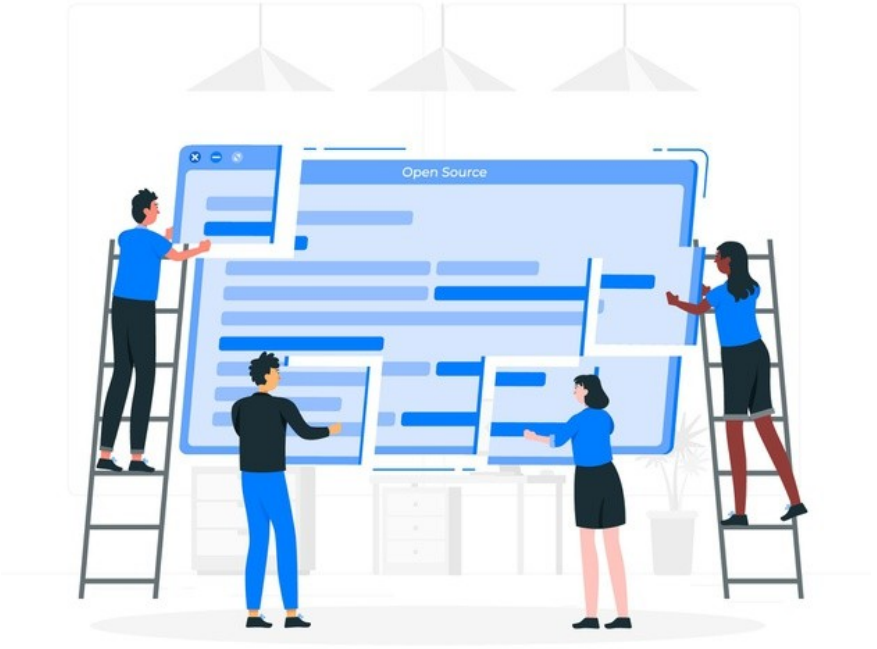
- « List of ordered and depending **tasks & milestones** needed to provide a given goal/achievement/product »
- Involves defining tasks, milestones, roles, timelines, and budgets. A well-crafted plan ensures clear communication, avoids roadblocks, and keeps projects on track.

Project Planning – GANTT CHart



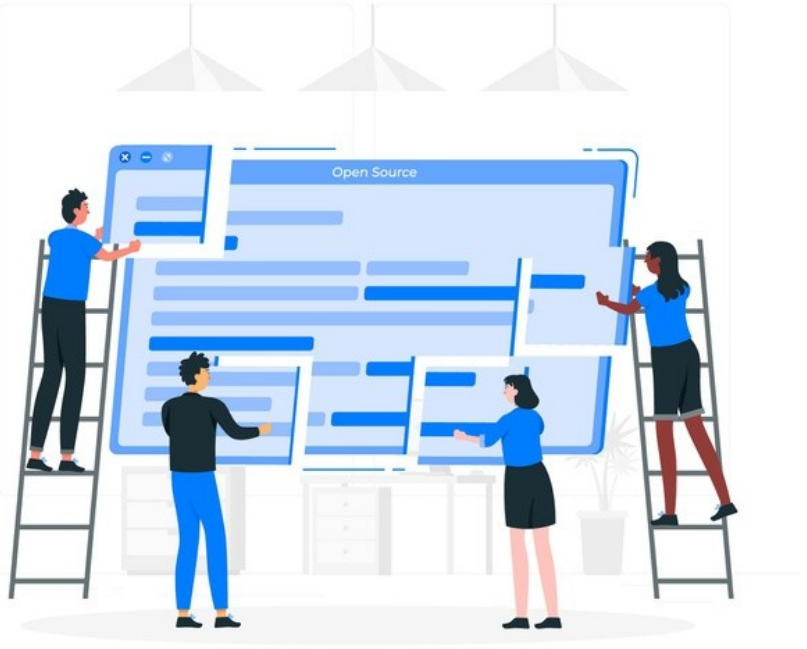
Development, Process, Product

Software Process :
« the set of rules, methods and techniques used during the development and maintainance of a software product



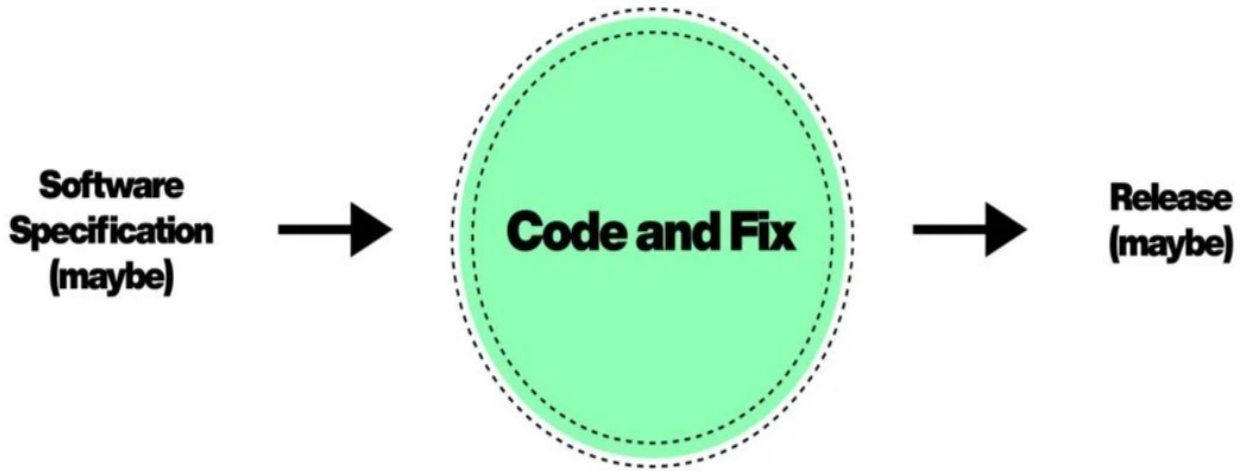
Software Processes

- **Code and Fix : if** you don't use any process
  - **Waterfall**: linear, **sequential** tasks, distinct phases
  - **RUP** (Rational Unified Process) **: iterative,** software process **framework** to be **tailored** for your needs
  - **Agile**: **flexible**, iterative, approach with rapid prototyping and **continuous delivery**.
  - **Scrum**: an Agile methodology that emphasizes **teamwork**, iterative dev, and **adaptive** planning.
- **DevOps**: improve collaboration between **dev** and **operations** teams, **automation** of software delivery.



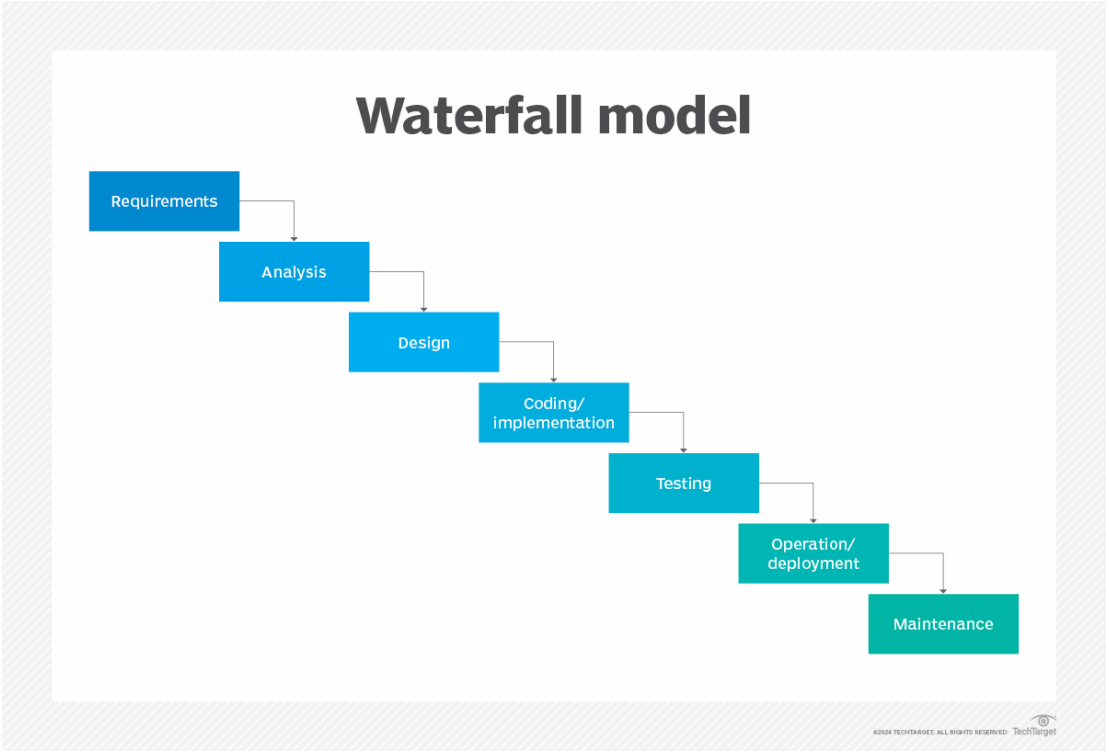
Code and Fix Process

- aka « **Cowboy coding** »
- Jump into coding
- Design while coding
- Fix problems on the way
- Coder has complete control of the design.



Waterfall Model

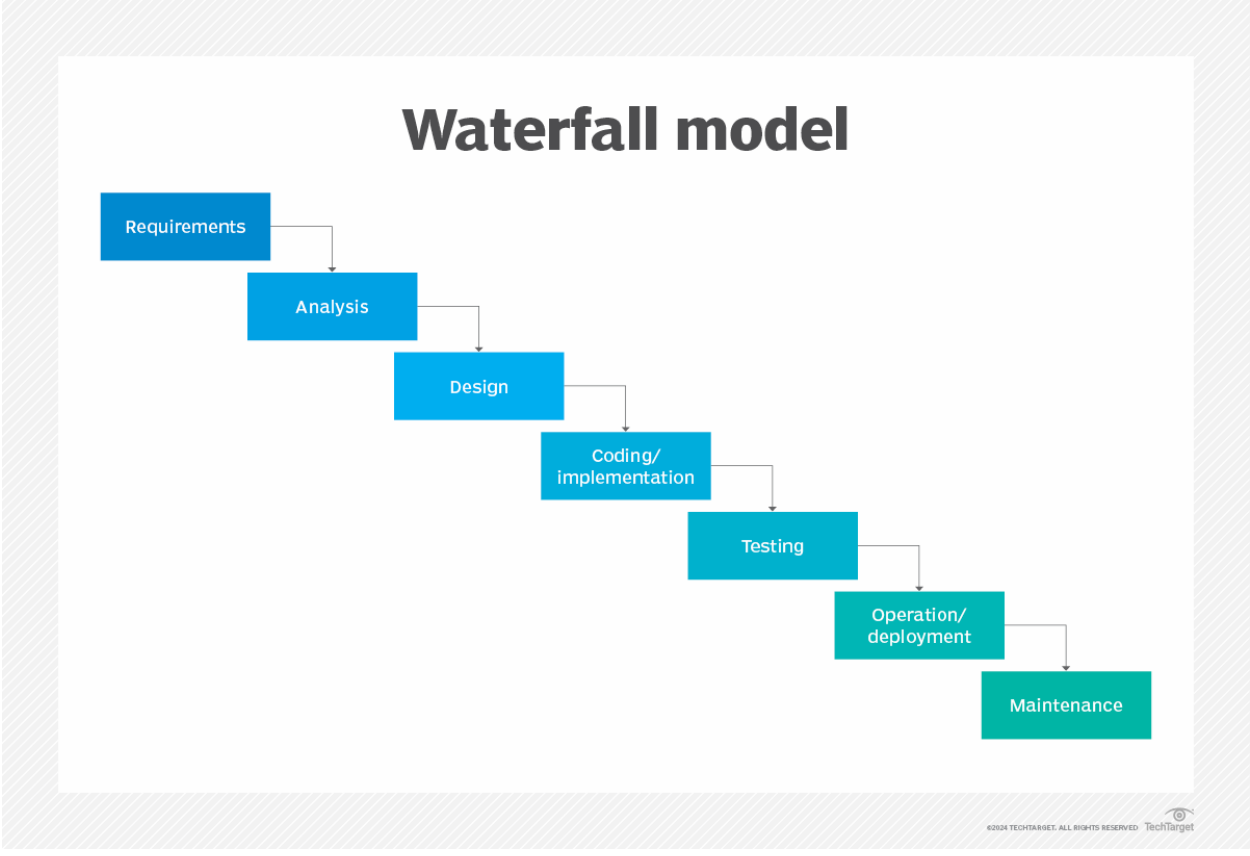
- each phase of the development cascades into the next, like how a waterfall flows.
- “linear sequential flow,” the outcome of one phase acts as the input for the next
- any stage in the development process begins **only** if the previous step is complete.
- **No going back**. Each stage relies on information from the previous stage and has its own project plan.



Waterfall Software Process

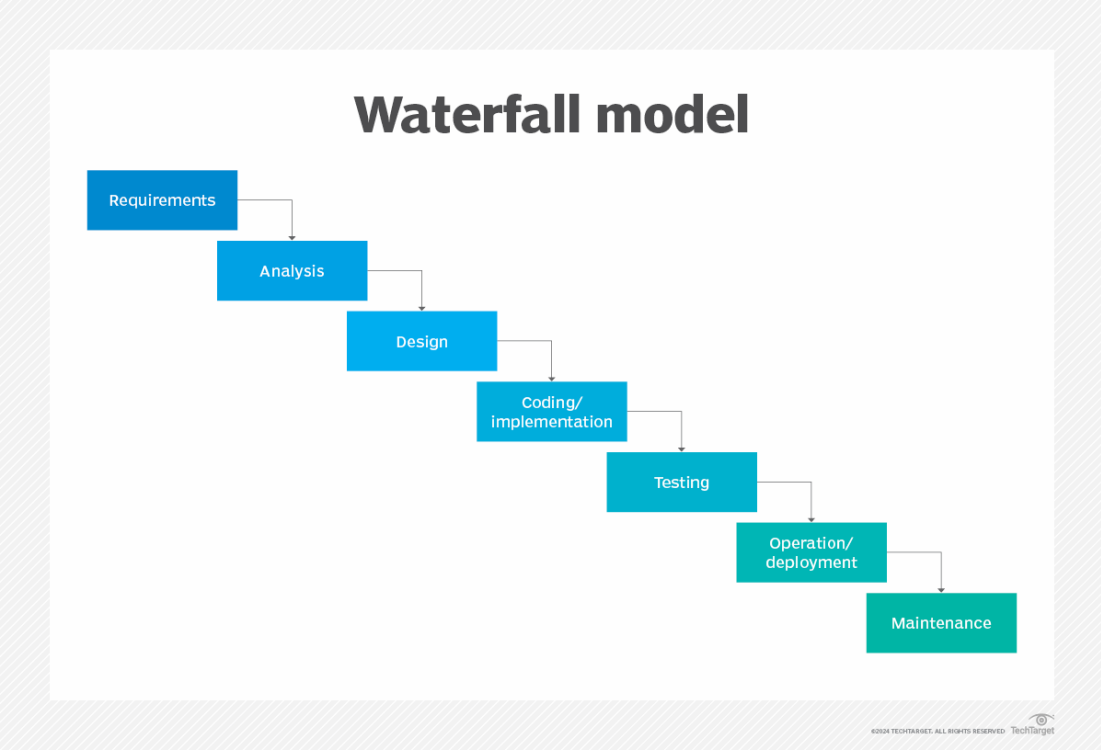
Phases validated by milestones :

- requirements gathering
- Analysis & Design
- Implementation
- Testing
- Maintenance



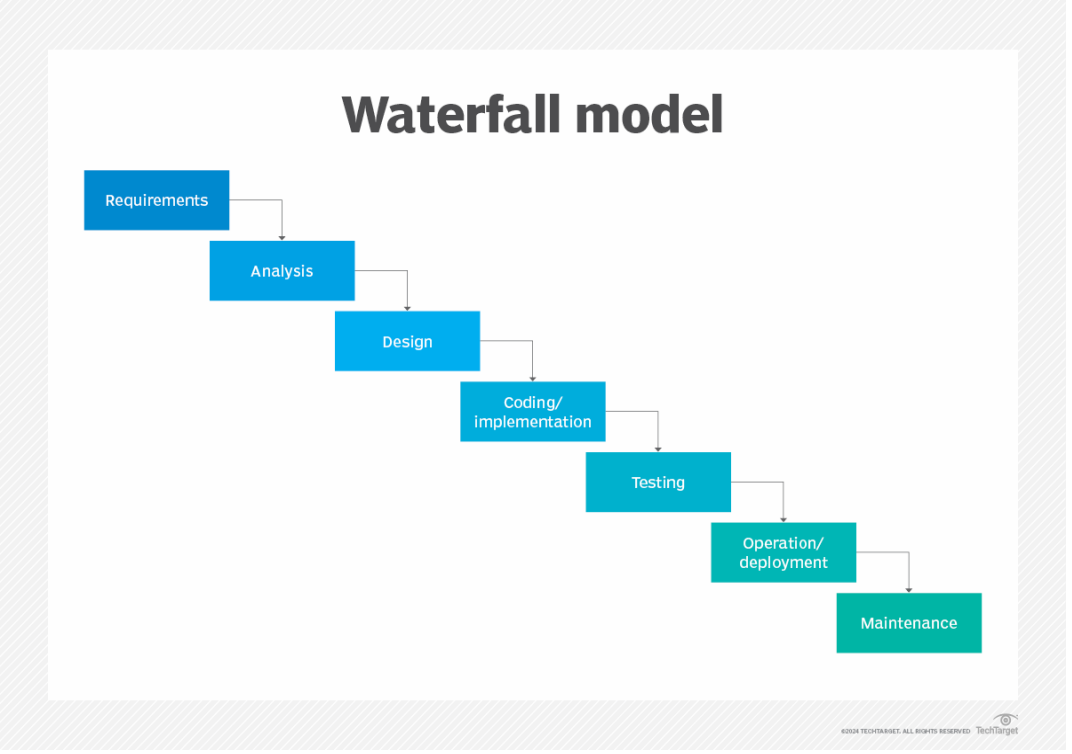
Process example – Waterfall Model

- **Specification** – Definition of business need and the desired behavior/output.
- **Design** – Specs laid out and documented, technical choices at low level
- **Development** – Writing code according to specs to produce the software.
- **Integration** – integrates with hardware and/or other software.
- **Testing** – evaluate if conform to specifications.
- **Installation/Deployment** – Delivery to the customer for installation and launching.
- **Maintenance** – deal with any issues raised, deliver updates or patches.



Waterfall Model - Limitations

- Nothing produced until late in the schedule
- Big risk and uncertainty.
- Not good for long complex projects.
- no coding before implementation.
- Difficult to measure progress.
- Cannot change requirements.
- Clients cannot validate early



Capability Maturity Model (SEI - CMM)

- a procedure to develop and refine an organization's software development process.
- defines a **five-level** evolutionary stage of increasingly organized and consistently more mature processes.
- CMM was developed and promoted by the **Software Engineering Institute (SEI)**, a research and development center promote by the U.S. Department of Defense (DOD).
- used as a benchmark to measure the maturity of an organization's software process.

Capability Maturity Model - Levels

- **Level 1: Initial**

  few or no processes are described and followed. Different engineers follow different process...as a result, development efforts are chaotic. It is called a **chaotic level.**

- **Level 2: Repeatable**

  fundamentals of project management practices - tracking cost, schedule - are established.

- **Level 3: Defined**

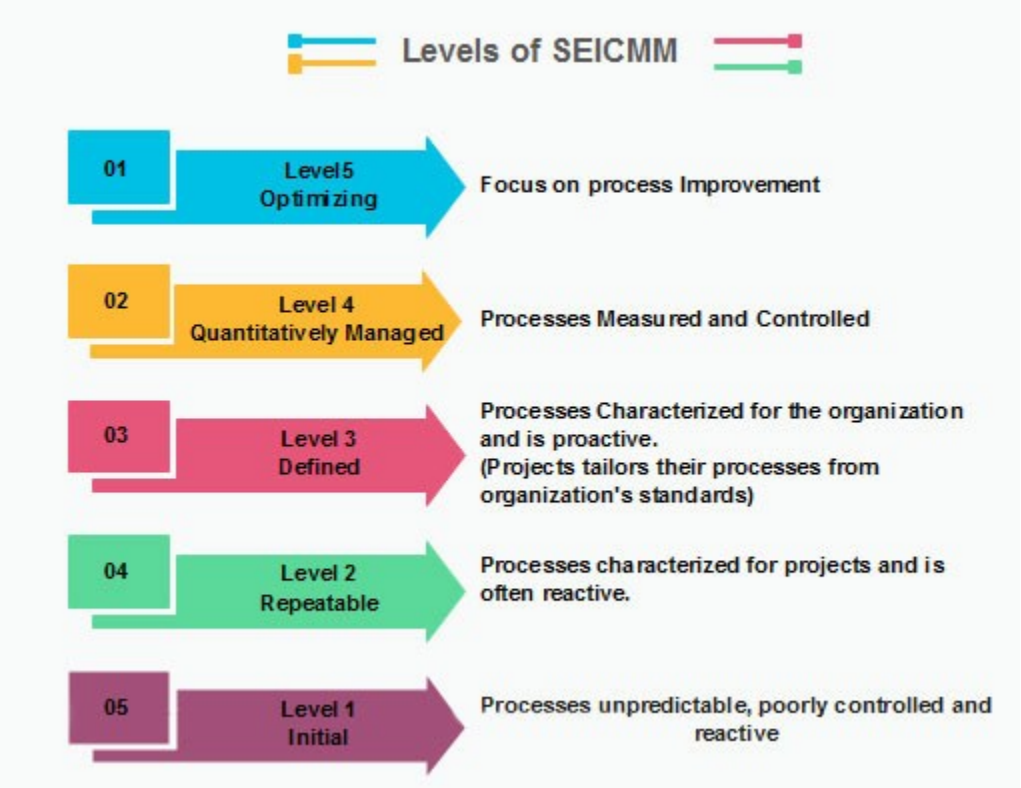
  methods for management and development are defined and documented. There is a common organization-wide understanding of operations, roles, and responsibilities. ISO 9000 goals at achieving this level.

- **Level 4: Managed**

  focus is on software metrics. **Product metrics** measure the features of the product being developed (size, reliability, complexity, performance, understandability... **Process metrics** measure the effectiveness of the process being used. The software process and product quality are measured, and quantitative quality requirements for the product are met.

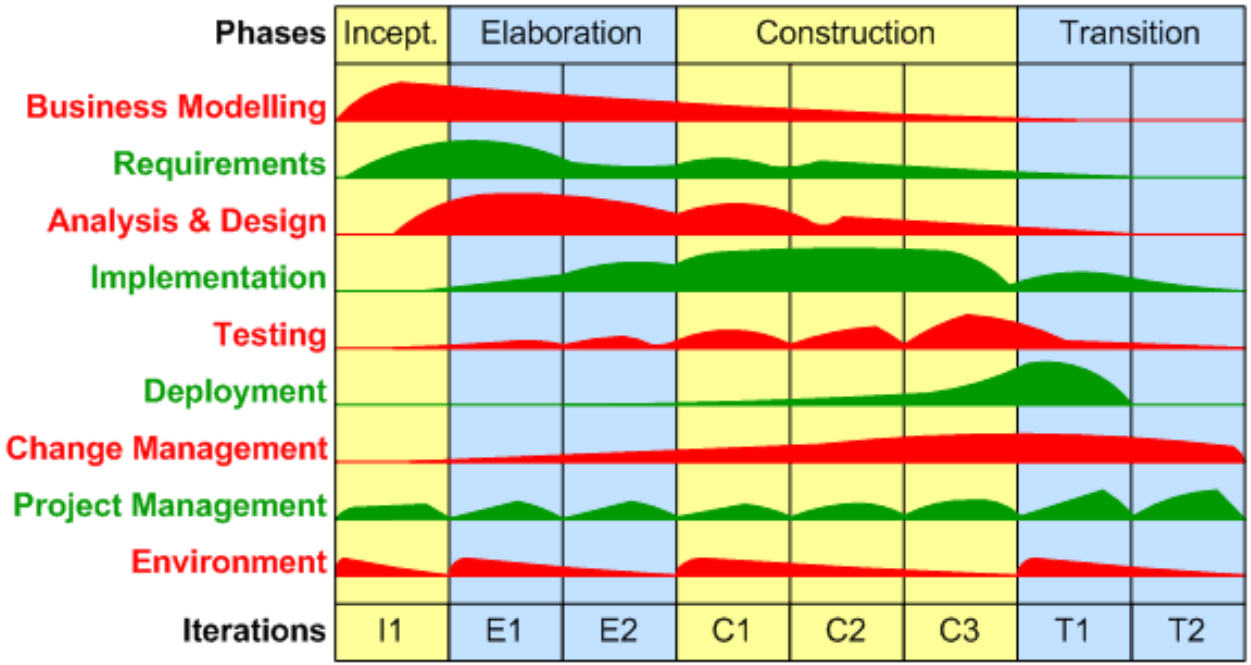
- **Level 5: Optimizing**

  process and product metrics are collected. Process and product measurement data are evaluated for continuous process improvement.

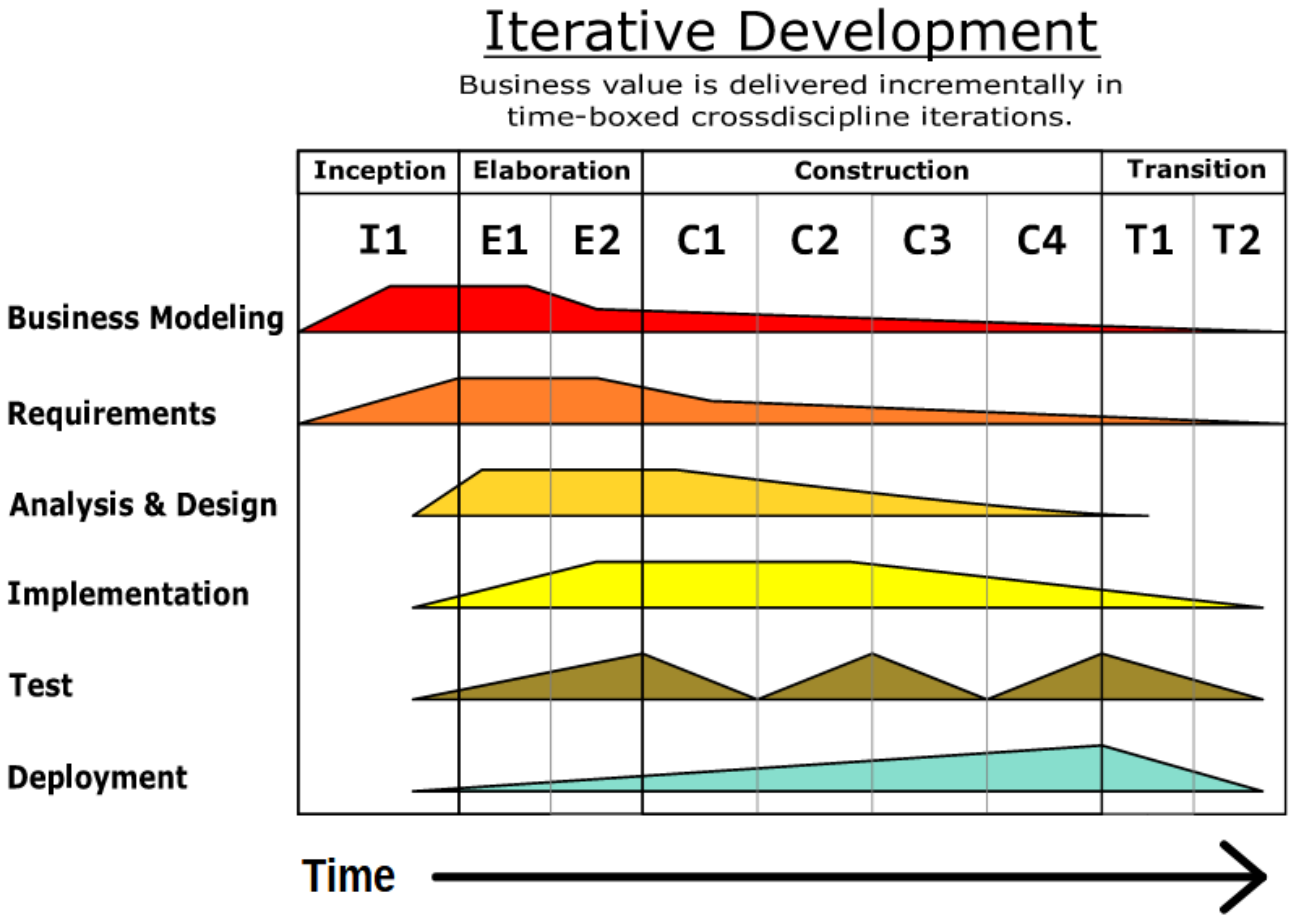


Rational Unified Process (RUP)

an **iterative** software development process **framework** created by the Rational Software Corporation (a division of IBM since 2003). RUP is not a single concrete prescriptive process, but rather an adaptable process framework, intended to be tailored by the software project team that will select the elements that are appropriate for their needs. RUP is a specific implementation of the **Unified Process** and uses the UML language

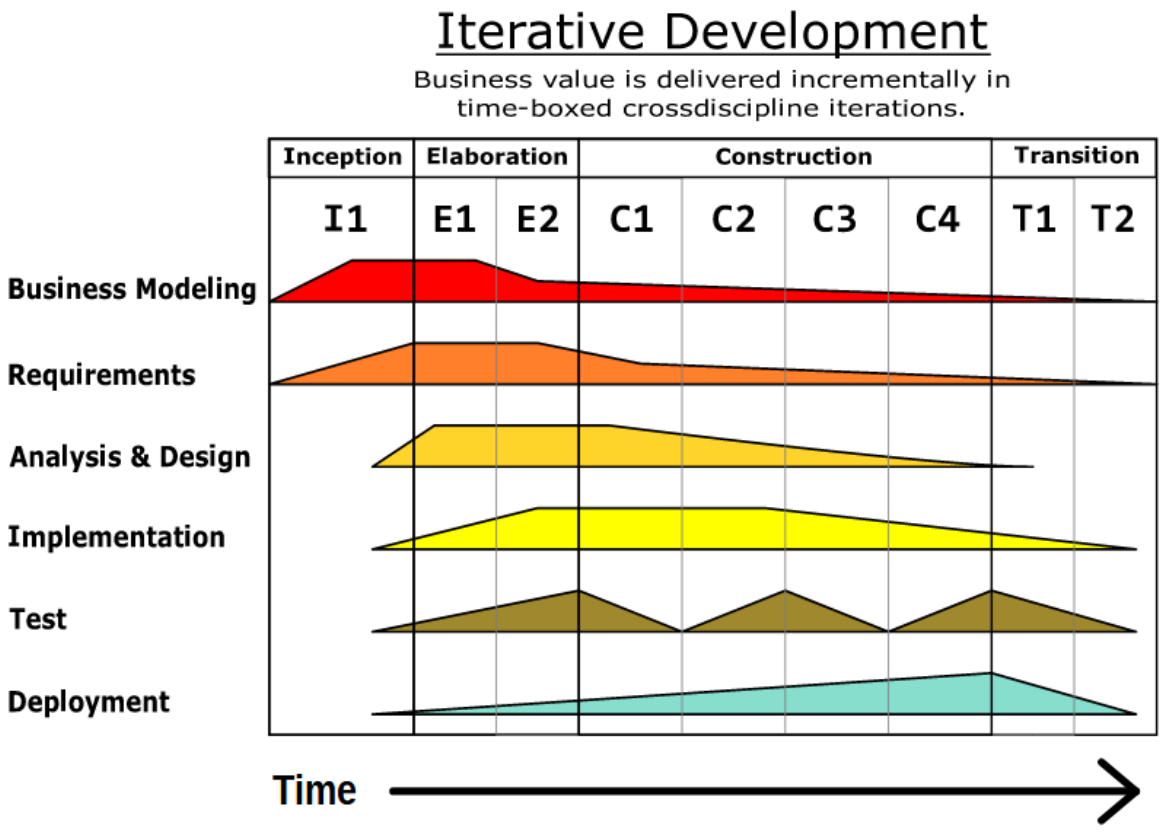


Rational Unified Process



Rational Unified Process

- **Advantages :**
- Acknowledges the impossibility to separate activities
- Aims at controlling software development (planning, cost quality)
- **Drawbacks:**
- A framework that needs to be customized



Agile Software Process

« Agile » Software Development

An umbrella term for approaches to developing software reflecting the values and principles agreed upon by The Agile Alliance, a group of 17 software practitioners in 2001. As documented in their Manifesto for Agile Software Development the practitioners value:

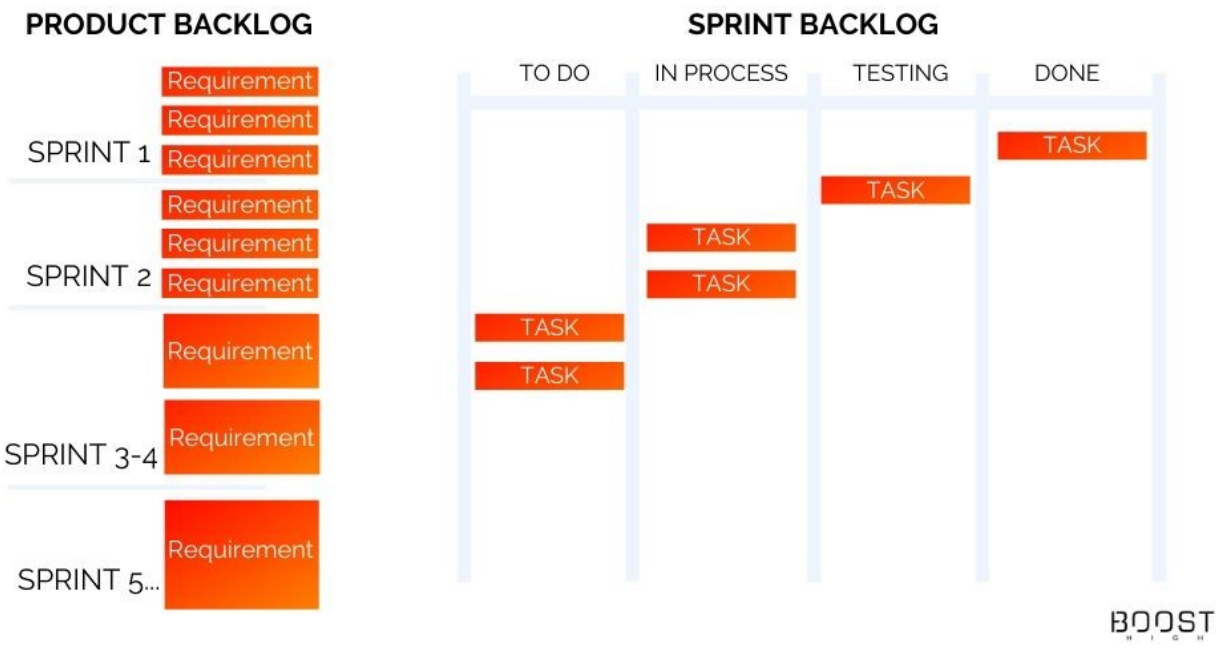
- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Software Process

- building products using short cycles of work. That allows rapid production and constant revision.
- stakeholders work together at each stage of the project. At each stage, each team goes through a process of planning, execution and evaluation.
- include collaborative effort of teams with their end-users, adaptive planning, evolutionary development, early delivery, continuous improvement, flexible responses to changes in requirements...

Agile Software Process - Backlog

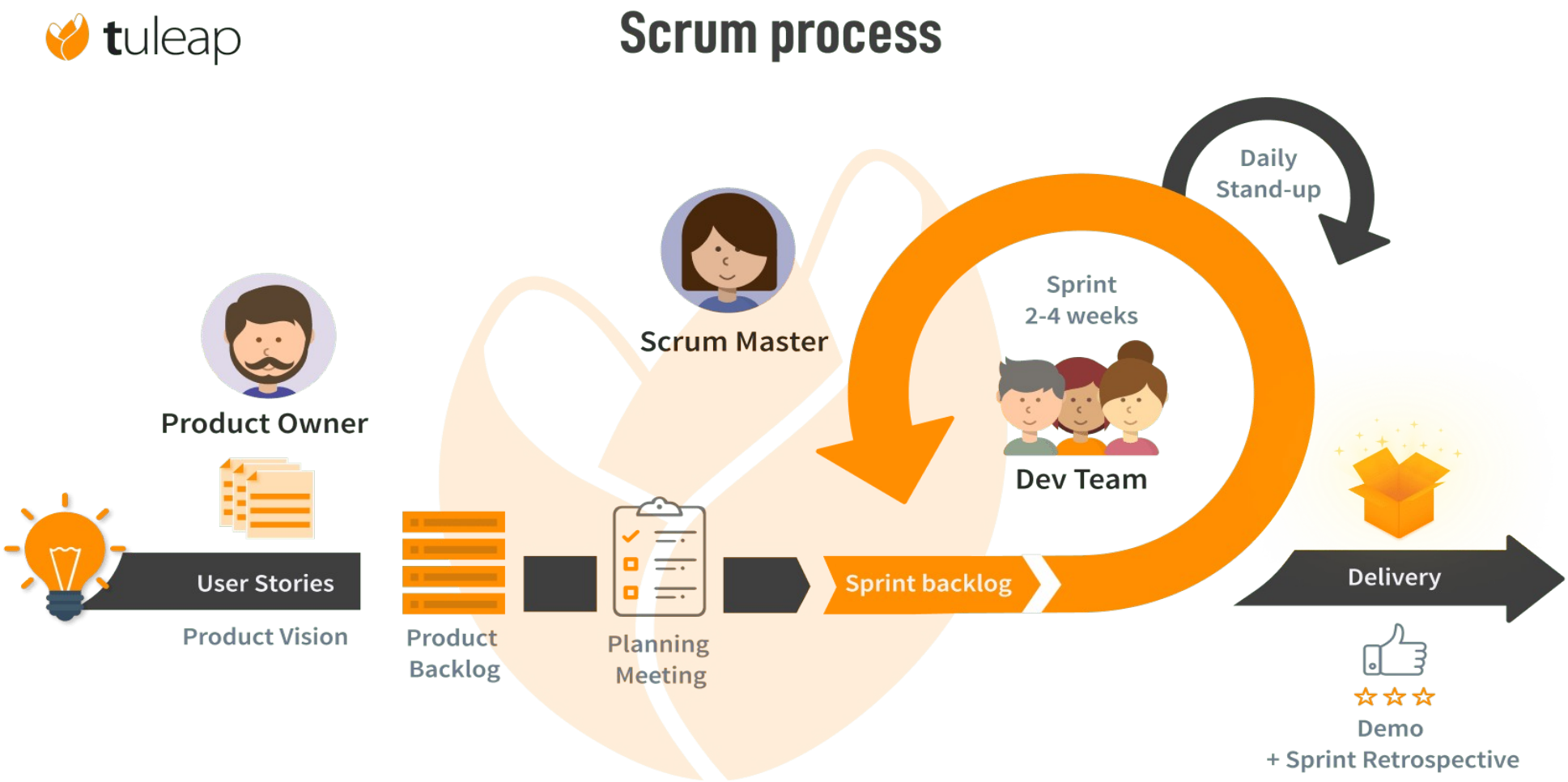
- A list of tasks, requirements, or features that still need to be completed or implemented. It is the essential tool for efficient planning and prioritizing tasks within a project or a development team.



Scrum

An **agile** project management framework that helps teams structure and manage their work through a set of values, principles, and **practices**. Much **like a rugby team** (where it gets its name) training for the big game, scrum encourages teams to learn through experiences, **self-organize** while working on a problem, and reflect on their wins and losses to continuously improve.

Scrum Agile Software Process



Scrum Agile

- Scrum allows teams to adjust to changes in the project during development **quickly**.
- Instead of following a strict order like waterfall, scrum uses short work periods called **sprints**.
- These sprints usually last two weeks and involve planning, designing, building, and testing the software to give users a working product.

Agile vs Scrum

While scrum is implemented at a product development team level, agile focuses on the entire organization, including its leadership and company culture. Many organizations use scrum in combination with other agile principles and practices to organize their teams.

Conclusion

- Select a Process or build your own
- Not defining a process is a default process !
- Stick to it
- State what you will do
- Do it
- Adapt & React to the Unexpected